

Hands-On Knowledge Graphs and Graph AI for Biomedical Research

From CSV files to a biomedical knowledge graph, graph algorithms, and an evidence-grounded AI agent

A One-Day Workshop

Srinath Jagannathan

Director & Lead Architect

Compute Elements Saranga Technologies

LinkedIn: [linkedin.com/in/srinathj](https://www.linkedin.com/in/srinathj)

From CSV files to a biomedical knowledge graph, graph algorithms, and an evidence-grounded AI agent

Neo4j + Graph Data Science + Cypher + LangGraph

It is an educational research workshop. We will build a graph, query it, analyze its structure, generate research hypotheses, and use an agent that shows its Cypher and evidence.

Today's journey

```
Curated CSV files
↓
Neo4j knowledge graph
↓
Cypher graph investigation
↓
Graph algorithms and link prediction
↓
AI research agent using Neo4j evidence
```

By the end, you will be able to:

- Build a Drug–Protein–Gene–Disease graph
- Ask questions using Cypher
- Identify important, similar, and bridging entities
- Generate **research hypotheses**, not medical advice
- Use an AI agent that retrieves graph evidence through Neo4j

Why connected data matters

Traditional data often lives in separate tables:

Data type	Example
Drugs	Metformin, Imatinib, Aspirin
Proteins	AMPK, ABL1, COX2
Genes	PRKAA1, ABL1, PTGS2
Diseases	Diabetes, CML, Inflammation
Pathways	Insulin signaling, DNA repair

The real research question is usually about the **connections**:

“Which drugs target proteins encoded by genes associated with a disease?”

What is a graph database?

A graph database stores data as:

- **Nodes** — entities such as drugs, genes, diseases, proteins, pathways
- **Relationships** — meaningful connections such as `TARGETS`, `TREATS`, or `ASSOCIATED_WITH`
- **Properties** — details such as evidence score, source, approval status, and year

```
(Drug) - [:TARGETS] -> (Protein) - [:ENCODED_BY] -> (Gene)  
(Gene) - [:ASSOCIATED_WITH] -> (Disease)
```

In a graph, relationships are first-class data—not hidden inside join tables.

Graphs make paths visible

```
Metformin
  | TARGETS
  ▼
AMPK Protein
  | ENCODED_BY
  ▼
PRKAA1 Gene
  | ASSOCIATED_WITH
  ▼
Type 2 Diabetes
```

A graph helps us see:

- **Direct connections** — known drug-treatment links
- **Indirect paths** — possible biological mechanisms
- **Shared neighbors** — common genes, targets, or pathways

Graph search can feel intelligent

A graph query can answer sophisticated questions without training a model:

```
MATCH path =  
  (drug:Drug)-[:TARGETS]->(:Protein)-[:ENCODED_BY]->(:Gene)  
  -[:ASSOCIATED_WITH]->(disease:Disease)  
WHERE disease.name = "Type 2 Diabetes Mellitus"  
RETURN path;
```

This is powerful because it asks for a **pattern**, not a spreadsheet row.

- “Find drugs connected to a disease through a biological mechanism”
- “Find diseases that share associated genes”
- “Find target overlap between drug classes”

Why graphs help AI and ML

Graph structure contains information beyond individual rows:

- **Who is important?** Centrality algorithms
- **Who is similar?** Similarity algorithms
- **Who forms a group?** Community detection
- **What may be missing?** Link prediction
- **How are two entities connected?** Path-finding

Machine learning can use graph features—but many insights appear before training a model.

From graph data to research insight

Data

- Knowledge graph
- Graph query
 - Graph algorithm
 - Research hypothesis
 - Evidence review and validation

Important distinction:

Appropriate use	Not appropriate
Research exploration	Diagnosis
Hypothesis generation	Treatment prescription
Evidence navigation	Dosage recommendation
Literature prioritization	Patient-specific advice

Our biomedical graph model

```
(:Drug)-[:TARGETS]->(:Protein)  
(:Protein)-[:ENCODED_BY]->(:Gene)  
(:Gene)-[:ASSOCIATED_WITH]->(:Disease)  
(:Drug)-[:TREATS]->(:Disease)  
(:Gene)-[:PARTICIPATES_IN]->(:Pathway)
```

Each important relationship includes:

- evidence_score
- evidence_type
- source

Example:

```
(Metformin)-[:TARGETS {action: "activates", evidence_score: 0.92}]->(AMPK)
```

Graph algorithms: the big idea

Graph algorithms analyze the **shape** of the network.

They answer questions that are hard to see by browsing nodes manually:

Research question	Algorithm family
What matters most?	Centrality
What connects separate groups?	Bridge / path algorithms
What looks alike?	Similarity
What forms a cluster?	Community detection
What relationship may be missing?	Link prediction

PageRank: “What is important?”

PageRank assigns importance based on connections and the importance of connected nodes.

A node gets a higher score when it is connected to many important nodes.

High PageRank could indicate:
Gene → associated with several diseases
Protein → targeted by several drugs
Pathway → connects several high-value genes

In our workshop:

- Rank influential genes, proteins, diseases, or pathways
- Identify high-priority entities for research review
- Show that importance is derived from **network structure**

Degree: “What is highly connected?”

Degree centrality counts the number of relationships attached to a node.

High degree = many direct connections

Examples in our graph:

- A gene linked to several diseases
- A protein targeted by several drugs
- A pathway containing many genes

Use degree when you need a quick answer to:

“Which entities are the hubs in this graph?”

Betweenness: “What is the bridge?”

Betweenness centrality measures how often a node lies on shortest paths between other nodes.

Cluster A — Bridge Node — Cluster B

High-betweenness entities can connect otherwise separate research areas.

In our workshop, it can help identify:

- Genes bridging oncology and inflammation
- Proteins connecting drugs to disease clusters
- Pathways that connect multiple disease areas

Think of betweenness as a transit hub or bridge in the knowledge network.

Closeness: “What is centrally positioned?”

Closeness centrality measures how near a node is to all other reachable nodes.

High closeness = fewer hops to reach the rest of the graph

In our research graph, high closeness may identify:

- Centrally positioned genes
- Efficient entry points for investigation
- Nodes near many disease, pathway, and drug neighborhoods

Use it to ask:

“From which node can we efficiently explore the network?”

Similarity: “What looks alike?”

Node similarity compares nodes based on shared neighbors or shared structure.

```
Disease A – Gene 1, Gene 2, Gene 3  
Disease B – Gene 1, Gene 2, Gene 4
```

Disease A and Disease B may be structurally similar because they share genes.

In our workshop, similarity helps discover:

- Diseases with shared genetic context
- Drugs with shared protein targets
- Genes with similar disease or pathway neighborhoods

Similarity suggests a research connection—not that two diseases are identical.

Community detection: “What forms a group?”

Community detection finds groups of nodes that are more connected to each other than to the rest of the graph.

Metabolic cluster:

Diabetes ↔ Insulin signaling ↔ INSR ↔ PRKAA1 ↔ Metformin

Oncology cluster:

CML ↔ ABL1 ↔ Imatinib ↔ Dasatinib

In our workshop, communities may reveal:

- Disease modules
- Drug-target neighborhoods
- Pathway-focused research clusters
- Shared biological themes

Shortest path: “How are they connected?”

Shortest-path algorithms find the smallest number of steps between two nodes.

Example question:

“How is Metformin connected to Type 2 Diabetes Mellitus?”

Metformin → AMPK Protein → PRKAA1 Gene → Type 2 Diabetes

Research uses:

- Explain a graph-derived result
- Find candidate biological mechanisms
- Reveal whether an entity connects directly or indirectly
- Support transparent agent responses

Link prediction: “What may be missing?”

Link prediction ranks possible relationships that are not currently present.

Known:
Drug A → Protein → Gene → Disease

Missing:
Drug A - - - > Disease

The graph may suggest a candidate association because the surrounding structure is similar to known relationships.

In our workshop:

- Find candidate Drug–Disease links
- Prioritize graph-supported hypotheses
- Inspect the supporting Protein–Gene–Disease path
- Require evidence review before any conclusion

A predicted link is a question to investigate—not a proven treatment.

Algorithms used in our workshop

Algorithm	What it answers	Workshop purpose
Degree centrality	What has many links?	Identify hubs
PageRank	What is structurally important?	Rank research entities
Node similarity	What shares a neighborhood?	Compare diseases or drugs
Louvain communities	What forms a cluster?	Reveal modules

Algorithms used in our workshop (continued)

Algorithm	What it answers	Workshop purpose
Shortest path / Cypher paths	How are entities connected?	Explain evidence
Common neighbors	What missing link is plausible?	Candidate discovery
Link prediction	What relationship could exist?	Research hypotheses

Our workshop flow

1. Start Neo4j with Docker
2. Import curated CSV data
3. Validate the graph
4. Explore paths with Cypher
5. Run graph algorithms
6. Generate candidate links
7. Ask the AI research agent
8. Present evidence-backed team findings

We move from raw files → connected knowledge → explainable insight.

Lab 0: Start the environment

Screen: Terminal

```
docker compose up -d  
docker compose ps
```

Check:

```
RETURN "Neo4j is ready" AS status;
```

Screen: Neo4j Browser

Open:

```
http://localhost:7474
```

Login:

```
Username: neo4j  
Password: clinicalkg2026
```

Lab 1: Create the graph rules

Screen: Neo4j Browser

Run `00_setup.cypher`

```
CREATE CONSTRAINT drug_id_unique IF NOT EXISTS  
FOR (n:Drug)  
  REQUIRE n.id IS UNIQUE;
```

We create:

- Unique IDs for every entity type
- Indexes for common searches
- A cleaner, safer import process

Constraints protect graph quality before loading data.

Lab 2: Import the CSV knowledge graph

Screen: Folder view

```
data/curated/  
├── drugs.csv  
├── diseases.csv  
├── genes.csv  
├── proteins.csv  
├── pathways.csv  
├── drug_targets_protein.csv  
├── gene_associated_disease.csv  
└── drug_treats_disease.csv
```

Screen: Neo4j Browser

Run `01_import_graph.cypher`

```
LOAD CSV WITH HEADERS FROM 'file:///drugs.csv' AS row  
MERGE (d:Drug {id: row.drug_id})  
SET d.name = row.drug_name;
```

Lab 2 activity: “Find the data model”

In pairs, answer before running the import:

1. Which node label represents a biomedical condition?
2. Which relationship captures a drug’s target?
3. Which property helps assess confidence?
4. Which property allows us to trace provenance?
5. Why is `TREATS` different from an indirect graph path?

Expected concepts:

`Disease` | `TARGETS` | `evidence_score` | `source` | Direct evidence vs hypothesis

Lab 3: Validate the graph

Screen: Neo4j Browser

```
MATCH (n)
RETURN labels(n) AS node_type, count(*) AS count
ORDER BY count DESC;
```

```
MATCH ()-[r]->()
RETURN type(r) AS relationship_type, count(*) AS count
ORDER BY count DESC;
```

Validation questions:

- Did all expected node labels load?
- Did all relationship types load?
- Do relationships include `source` ?
- Can we see a Drug → Protein → Gene → Disease path?

Lab 3: Explore evidence paths

Screen: Neo4j Browser

```
MATCH path =  
  (drug:Drug {name: "Metformin"})-[:TARGETS]->(:Protein)  
  -[:ENCODED_BY]->(gene:Gene)  
  -[r:ASSOCIATED_WITH]->(disease:Disease {  
    name: "Type 2 Diabetes Mellitus"  
  })  
RETURN path,  
  gene.symbol AS supporting_gene,  
  r.evidence_score AS evidence_score,  
  r.source AS source;
```

```
</div>  
<div>
```

Discuss:

- What biological route did the graph return?
- What source supports the association?
- Is this a treatment claim or a mechanistic path?

Lab 3 activity: Research detective

Each team chooses one disease:

- Type 2 Diabetes Mellitus
- Breast Cancer
- Chronic Myeloid Leukemia
- Hypertension
- Rheumatoid Arthritis
- Lung Cancer

Team task:

1. Find associated genes
2. Find related pathways
3. Find drugs with a graph path to the disease
4. Record evidence source and score
5. Present one graph path visually

Your output is an evidence trail—not a clinical recommendation.

Lab 4: Create a GDS projection

Screen: Neo4j Browser

```
CALL gds.graph.project(  
  'clinicalGraph',  
  ['Drug', 'Protein', 'Gene', 'Disease', 'Pathway'],  
  {  
    TARGETS: {orientation: 'UNDIRECTED'},  
    ENCODED_BY: {orientation: 'UNDIRECTED'},  
    ASSOCIATED_WITH: {orientation: 'UNDIRECTED'},  
    TREATS: {orientation: 'UNDIRECTED'},  
    PARTICIPATES_IN: {orientation: 'UNDIRECTED'}  
  }  
)  
YIELD graphName, nodeCount, relationshipCount  
RETURN graphName, nodeCount, relationshipCount;
```

Lab 4: Rank influential entities

Screen: Neo4j Browser

```
CALL gds.pageRank.stream('clinicalGraph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS node, score
RETURN
  coalesce(node.name, node.symbol, node.id) AS entity,
  labels(node) AS entity_type,
  round(score, 4) AS pagerank
ORDER BY pagerank DESC
LIMIT 20;
```

Before running it, vote:

Which will rank highest: a drug,
gene, protein, disease, or pathway?

Discuss after result:

- Is a high score caused by biology, data coverage, or both?
- What additional evidence would you need?
- How should a researcher use this ranking?

Lab 4: Compare disease neighborhoods

Screen: Neo4j Browser

```
CALL gds.nodeSimilarity.stream(  
  'clinicalGraph',  
  {  
    nodeLabels: ['Disease'],  
    relationshipTypes: ['ASSOCIATED_WITH'],  
    similarityCutoff: 0.1  
  }  
)  
YIELD node1, node2, similarity  
RETURN  
  gds.util.asNode(node1).name AS disease_1,  
  gds.util.asNode(node2).name AS disease_2,  
  round(similarity, 3) AS similarity  
ORDER BY similarity DESC  
LIMIT 20;
```

Interpretation:

- Similarity means shared graph context
- Shared context may mean common genes or pathways
- Similarity does **not** mean identical disease biology

Optional: Find bridge entities

Screen: Neo4j Browser

```
CALL gds.betweenness.stream('clinicalGraph')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS node, score
RETURN
    coalesce(node.name, node.symbol, node.id) AS entity,
    labels(node) AS entity_type,
    round(score, 2) AS betweenness
ORDER BY betweenness DESC
LIMIT 20;
```

Prompt:

Which high-betweenness entity could connect multiple disease communities?

Use cases:

Lab 5: Candidate link discovery

Screen: Neo4j Browser

```
MATCH (drug:Drug)-[:TARGETS]->(:Protein)-[:ENCODED_BY]->(gene:Gene)
MATCH (gene)-[assoc:ASSOCIATED_WITH]->(disease:Disease)
WHERE NOT (drug)-[:TREATS]->(disease)
RETURN
  drug.name AS candidate_drug,
  disease.name AS candidate_disease,
  count(DISTINCT gene) AS supporting_gene_count,
  round(avg(assoc.evidence_score), 3) AS mean_evidence_score,
  collect(DISTINCT gene.symbol)[0..8] AS supporting_genes
ORDER BY supporting_gene_count DESC, mean_evidence_score DESC
LIMIT 20;
```

Output = a graph-supported candidate research hypothesis.

Lab 5 activity: Research hypothesis market

Each team receives **100 fictional research credits**.

Allocate credits across five candidate Drug–Disease pairs.

For your highest-ranked candidate, provide:

1. Candidate drug and disease
2. Supporting genes
3. Visible evidence path
4. Evidence score and source
5. One limitation
6. Next validation step

The AI agent: why add it?

Cypher is powerful, but not every researcher writes Cypher.

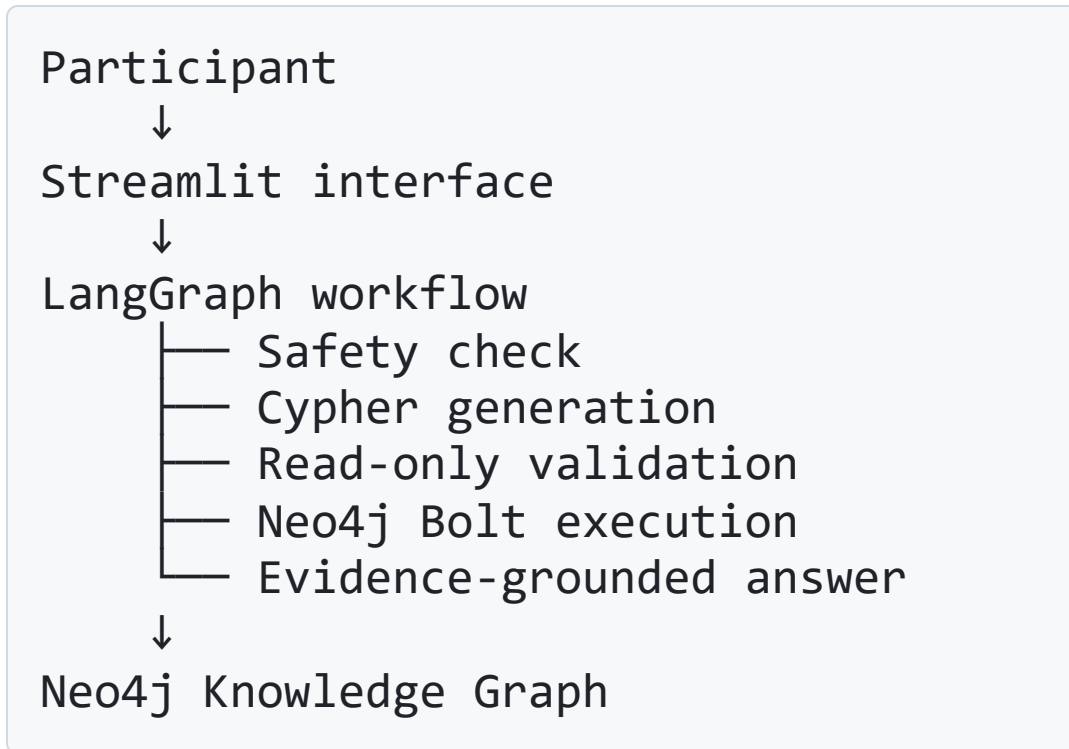
The agent lets participants ask:

“Which drugs connect to Type 2 Diabetes through proteins and genes, excluding known treatments?”

The agent workflow:

```
graph TD; A[Natural-language question] --> B[Safety and research-intent check]; B --> C[LLM generates read-only Cypher]; C --> D[Cypher validator]; D --> E[Neo4j Bolt query execution];
```

Agent architecture



Safety controls:

- Read-only **MATCH** queries only
- No **CREATE**, **DELETE**, **MERGE**, **SET**, or **CALL**
- Maximum result limit
- Shows generated Cypher
- Shows returned records and sources
- Refuses diagnosis, dosage, and patient-specific treatment requests

Agent live demo

Screen: Streamlit research agent

Ask:

Which genes are associated with Type 2 Diabetes Mellitus, and what evidence supports them?

Show participants:

1. The natural-language question
2. The generated Cypher
3. Returned Neo4j records
4. Evidence scores and sources
5. The concise research response

Agent transparency example

Screen: Generated Cypher

```
MATCH (gene:Gene)-[r:ASSOCIATED_WITH]->(disease:Disease)
WHERE disease.name = "Type 2 Diabetes Mellitus"
RETURN
  gene.symbol AS gene,
  gene.name AS gene_name,
  r.evidence_score AS evidence_score,
  r.evidence_type AS evidence_type,
  r.source AS source
ORDER BY evidence_score DESC
LIMIT 30;
```

The agent should say:

- What the graph returned
- Which evidence supports it

Agent challenge

Ask the agent a research question of your own.

Good questions:

- "Show a graph path from Imatinib to Chronic Myeloid Leukemia."
- "Which diseases share inflammatory genes?"
- "Which drugs target proteins linked to Lung Cancer?"
- "Which candidate drug-disease pairs are indirectly connected but not marked as treatments?"

Then inspect:

- Did it generate safe Cypher?
- Did it retrieve useful graph evidence?
- Did it state sources?
- Did it describe indirect links as hypotheses?

What the agent must refuse

Screen: Streamlit research agent

Ask:

What medicine should I take for diabetes?

Expected response:

I cannot provide diagnosis, medication, dosage, or patient-specific treatment advice. I can help explore educational biomedical research relationships in this graph.

Why this matters:

- A graph can support research navigation
- An AI agent can summarize retrieved evidence

Team presentation template

Each team presents in 2 minutes:

1. Research question
2. Disease or research domain
3. Graph path or key graph finding
4. Algorithm result or candidate link
5. Evidence source and score
6. One limitation
7. Suggested next validation step

Use this final sentence:

“Our graph suggests this is a research hypothesis worth investigating because...”

Key takeaways

- Graph databases make relationships directly queryable
- Cypher finds meaningful multi-hop evidence paths
- Graph algorithms reveal importance, bridges, similarity, and communities
- Link prediction produces candidates for further research
- AI agents make graph knowledge more accessible
- Transparency, provenance, and safety controls are essential

Graph + algorithms + evidence-grounded agent = a research navigation system.

Thank you

Questions and discussion

Suggested next steps

- Add more curated data sources
- Add publication and clinical-trial evidence nodes
- Improve evidence provenance
- Build richer GDS pipelines
- Add GraphRAG retrieval for unstructured literature
- Evaluate agent answers with expert review

CSV → Knowledge Graph → Graph Data Science → Evidence-Grounded AI